

Go from Data to Strategy ▶

ONLINE MASTER OF SCIENCE IN
BUSINESS ANALYTICSCarnegie Mellon University
Tepper School of Business

Go from Data to Strategy: Tepper School of Business

Understand Model Behavior During Training by Visualizing Metrics

by **Adrian Tam** on April 8, 2023 in **Deep Learning with PyTorch** 0

Share

Share

Post

You can learn a lot about neural networks and deep learning models by observing their performance over time during training. For example, if you see the training accuracy went worse with training epochs, you know you have issue with the optimization. Probably your learning rate is too fast. In this post, you will discover how you can review and visualize the performance of PyTorch models over time during training. After completing this post, you will know:

- What metrics to collect during training
- How to plot the metrics on training and validation datasets from training
- How to interpret the plot to tell about the model and training progress

Kick-start your project with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started.



Understand Model Behavior During Training by Visualizing Metrics
Photo by Alison Pang. Some rights reserved.

Overview

This chapter is in two parts; they are:

- Collecting Metrics from a Training Loop
- Plotting the Training History

Collecting Metrics from a Training Loop

In deep learning, training a model with gradient descent algorithm means to take a forward pass to infer loss metric from the input using the model and a loss function, then a backward pass to compute the gradient from the loss metric, and a update process to apply the gradient to update the model parameters. While these are the basic steps you must take, you can do a bit more along the process to collect additional information.

A model that trained correctly should expect the loss metric to decrease, as the loss is the objective to optimize. The loss metric to use should depends on the problem.

For regression problems, the closer the model's prediction to the actual value the better. Therefore you want to keep track on the mean square error (MSE), or sometimes root mean square error (RMSE), mean absolute error (MAE), or mean absolute percentage error (MAPE). Although not used as a loss metric, you may also interested in the maximum error produced by your model.

For classification problems, usually the loss metric is cross entropy. But the value of cross entropy is not very intuitive. Therefore you may also want to keep track on the accuracy of prediction, true positive rate, precision, recall, F1 scores, and so on.

Collecting these metrics from a training loop is trivial. Let's start with a basic regression example of deep learning using PyTorch with the California housing dataset:

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from sklearn.datasets import fetch_california_housing
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler
7
8 # Read data
9 data = fetch_california_housing()
10 X, y = data.data, data.target
11
12 # train-test split for model evaluation
13 X_train_raw, X_test_raw, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)
14
15 # Standardizing data
16 scaler = StandardScaler()
17 scaler.fit(X_train_raw)
18 X_train = scaler.transform(X_train_raw)
19 X_test = scaler.transform(X_test_raw)
20
21 # Convert to 2D PyTorch tensors
22 X_train = torch.tensor(X_train, dtype=torch.float32)
23 y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
24 X_test = torch.tensor(X_test, dtype=torch.float32)
25 y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)
26
27 # Define the model
28 model = nn.Sequential(
29     nn.Linear(8, 24),
30     nn.ReLU(),
31     nn.Linear(24, 12),
32     nn.ReLU(),
33     nn.Linear(12, 6),
34     nn.ReLU(),
35     nn.Linear(6, 1)
36 )
37
38 # loss function and optimizer
39 loss_fn = nn.MSELoss() # mean square error
40 optimizer = optim.Adam(model.parameters(), lr=0.001)
41
42 n_epochs = 100 # number of epochs to run
43 batch_size = 32 # size of each batch
44 batch_start = torch.arange(0, len(X_train), batch_size)
45
46 for epoch in range(n_epochs):
47     for start in batch_start:
48         # take a batch
49         X_batch = X_train[start:start+batch_size]
50         y_batch = y_train[start:start+batch_size]
51         # forward pass
52         y_pred = model(X_batch)
53         loss = loss_fn(y_pred, y_batch)
54         # backward pass
55         optimizer.zero_grad()
56         loss.backward()
57         # update weights
58         optimizer.step()
```

This implementation is primitive, but you obtained loss as a tensor in each step in the process which provides hints to the optimizer to improve the model. To know about the progress of the training, you can, of course, print this loss metric at every step. But you can also save this value so you can visualize it later. When you do that, beware that you do not want to save a tensor but simply its value. It is because the PyTorch tensor here remembers how it comes with its value so automatic differentiation can be done. These additional data are occupying memory but you do not need them.

Hence you can modify the training loop to the following:

```
1 mse_history = []
2
3 for epoch in range(n_epochs):
```

```

4     for start in batch_start:
5         # take a batch
6         X_batch = X_train[start:start+batch_size]
7         y_batch = y_train[start:start+batch_size]
8         # forward pass
9         y_pred = model(X_batch)
10        loss = loss_fn(y_pred, y_batch)
11        mse_history.append(float(loss))
12        # backward pass
13        optimizer.zero_grad()
14        loss.backward()
15        # update weights
16        optimizer.step()

```

In training a model, you should evaluate it with a test set which is segregated from the training set. Usually it is done once in an epoch, after all the training steps in that epoch. The test result can also be saved for visualization later. In fact, you can obtain multiple metrics from the test set if you want to. Hence you can add to the training loop as follows:

```

1 mae_fn = nn.L1Loss() # create a function to compute MAE
2 train_mse_history = []
3 test_mse_history = []
4 test_mae_history = []
5
6 for epoch in range(n_epochs):
7     model.train()
8     for start in batch_start:
9         # take a batch
10        X_batch = X_train[start:start+batch_size]
11        y_batch = y_train[start:start+batch_size]
12        # forward pass
13        y_pred = model(X_batch)
14        loss = loss_fn(y_pred, y_batch)
15        train_mse_history.append(float(loss))
16        # backward pass
17        optimizer.zero_grad()
18        loss.backward()
19        # update weights
20        optimizer.step()
21    # validate model on test set
22    model.eval()
23    with torch.no_grad():
24        y_pred = model(X_test)
25        mse = loss_fn(y_pred, y_test)
26        mae = mae_fn(y_pred, y_test)
27        test_mse_history.append(float(mse))
28        test_mae_history.append(float(mae))

```

You can define your own function to compute the metrics or use one that already implemented from PyTorch library. It is a good practice to switch the model to evaluation mode on evaluation. It is also good practice to run the evaluation under the `no_grad()` context, in which you explicitly tell PyTorch that you have no intention to run automatic differentiation on the tensors.

However, there is a problem in the code above: The MSE from training set is computed once per training step based on one batch while the metrics from the test set are computed once per epoch and based on the entire test set. They are not directly comparable. In fact, if you look at the MSE from training steps, you will find it **very noisy**. The better way is to summarize the MSE from the same epoch to one number (e.g., their mean) so you can compare to the test set's data.

Making this change, following is the complete code:

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from sklearn.datasets import fetch_california_housing
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler
7
8 # Read data
9 data = fetch_california_housing()
10 X, y = data.data, data.target
11
12 # train-test split for model evaluation
13 X_train_raw, X_test_raw, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)
14
15 # Standardizing data
16 scaler = StandardScaler()
17 scaler.fit(X_train_raw)
18 X_train = scaler.transform(X_train_raw)
19 X_test = scaler.transform(X_test_raw)
20
21 # Convert to 2D PyTorch tensors
22 X_train = torch.tensor(X_train, dtype=torch.float32)
23 y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
24 X_test = torch.tensor(X_test, dtype=torch.float32)
25 y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)
26
27 # Define the model
28 model = nn.Sequential(
29     nn.Linear(8, 24),
30     nn.ReLU(),
31     nn.Linear(24, 12),
32     nn.ReLU(),

```

```

33     nn.Linear(12, 6),
34     nn.ReLU(),
35     nn.Linear(6, 1)
36 )
37
38 # loss function, metrics, and optimizer
39 loss_fn = nn.MSELoss() # mean square error
40 mae_fn = nn.L1Loss() # mean absolute error
41 optimizer = optim.Adam(model.parameters(), lr=0.001)
42
43 n_epochs = 100 # number of epochs to run
44 batch_size = 32 # size of each batch
45 batch_start = torch.arange(0, len(X_train), batch_size)
46
47 train_mse_history = []
48 test_mse_history = []
49 test_mae_history = []
50
51 for epoch in range(n_epochs):
52     model.train()
53     epoch_mse = []
54     for start in batch_start:
55         # take a batch
56         X_batch = X_train[start:start+batch_size]
57         y_batch = y_train[start:start+batch_size]
58         # forward pass
59         y_pred = model(X_batch)
60         loss = loss_fn(y_pred, y_batch)
61         epoch_mse.append(float(loss))
62         # backward pass
63         optimizer.zero_grad()
64         loss.backward()
65         # update weights
66         optimizer.step()
67     mean_mse = sum(epoch_mse) / len(epoch_mse)
68     train_mse_history.append(mean_mse)
69     # validate model on test set
70     model.eval()
71     with torch.no_grad():
72         y_pred = model(X_test)
73         mse = loss_fn(y_pred, y_test)
74         mae = mae_fn(y_pred, y_test)
75         test_mse_history.append(float(mse))
76         test_mae_history.append(float(mae))

```

Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

Plotting the Training History

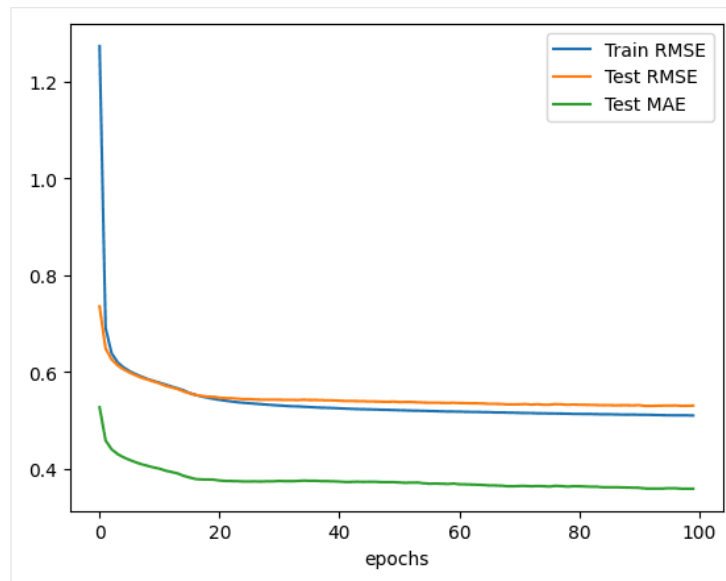
In the code above, you collected metrics in a Python list, one each per epoch. Therefore, it is trivial to plot them into a line graph using matplotlib. Below is an example:

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 plt.plot(np.sqrt(train_mse_history), label="Train RMSE")
5 plt.plot(np.sqrt(test_mse_history), label="Test RMSE")
6 plt.plot(test_mae_history, label="Test MAE")
7 plt.xlabel("epochs")
8 plt.legend()
9 plt.show()

```

It plots, for example, the following:



Plots like this can provide an indication of useful things about the training of the model, such as:

- Its speed of convergence over epochs (slope)
- Whether the model may have already converged (plateau of the line)
- Whether the model may be over-learning the training data (inflection for validation line)

In a regression example like the above, the metrics MAE and MSE should both decrease if the model gets better. In a classification example, however, accuracy metric should increase while the cross entropy loss should decrease as more training has been done. This is what you are expected to see from the plot.

These curves should eventually flatten, meaning you cannot improve the model any further based on the current dataset, model design, and algorithms. You want this to happen as soon as possible, so your model **converge** faster as your training is efficient. You also want the metric to flatten at a high accuracy or low loss region, so your model is effective in prediction.

The other property to watch for in the plots is how different are the metrics from training and validation. In the above, you see the training set's RMSE is higher than test set's RMSE at the beginning but very soon, the curves crossed and the test set's RMSE is higher at the end. This is expected, as eventually the model will fit better to the training set but it is the test set that can predict how the model performs on future, unseen data.

You need to be careful to interpret the curves or metrics in a microscopic scale. In the plot above, you see that the training set's RMSE is extremely large compare to that of test set's in epoch 0. Their difference may not be that drastic, but since you collected the training set's RMSE by taking the MSE of each steps during the first epoch, your model probably not doing well in the first few steps but much better at the last few steps of the epoch. Taking average across all the steps may not be a fair comparison as the MSE from test set is based on the model after the last step.

Your model is **overfit** if you see the training set's metric is much better than that from test set. This can hint that you should stop your training at an earlier epoch or your model's design need some regularization, such as dropout layer.

In the plot above, while you collected mean square error (MSE) for the regression problem but you plotted root mean square error (RMSE) instead, so you can compare to the mean absolute error (MAE) in the same scale. Probably you should also collect the MAE of the training set as well. The two MAE curves should behave similarly to that of the RMSE curves.

Putting everything together, the following is the complete code:

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 from sklearn.datasets import fetch_california_housing
7 from sklearn.model_selection import train_test_split
8 from sklearn.preprocessing import StandardScaler
9
10 # Read data
11 data = fetch_california_housing()
12 X, y = data.data, data.target
13
14 # train-test split for model evaluation
15 X_train_raw, X_test_raw, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)
16
17 # Standardizing data
18 scaler = StandardScaler()
19 scaler.fit(X_train_raw)
```

```

20 X_train = scaler.transform(X_train_raw)
21 X_test = scaler.transform(X_test_raw)
22
23 # Convert to 2D PyTorch tensors
24 X_train = torch.tensor(X_train, dtype=torch.float32)
25 y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
26 X_test = torch.tensor(X_test, dtype=torch.float32)
27 y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)
28
29 # Define the model
30 model = nn.Sequential(
31     nn.Linear(8, 24),
32     nn.ReLU(),
33     nn.Linear(24, 12),
34     nn.ReLU(),
35     nn.Linear(12, 6),
36     nn.ReLU(),
37     nn.Linear(6, 1)
38 )
39
40 # loss function, metrics, and optimizer
41 loss_fn = nn.MSELoss() # mean square error
42 mae_fn = nn.L1Loss() # mean absolute error
43 optimizer = optim.Adam(model.parameters(), lr=0.001)
44
45 n_epochs = 100 # number of epochs to run
46 batch_size = 32 # size of each batch
47 batch_start = torch.arange(0, len(X_train), batch_size)
48
49 train_mse_history = []
50 test_mse_history = []
51 test_mae_history = []
52
53 for epoch in range(n_epochs):
54     model.train()
55     epoch_mse = []
56     for start in batch_start:
57         # take a batch
58         X_batch = X_train[start:start+batch_size]
59         y_batch = y_train[start:start+batch_size]
60         # forward pass
61         y_pred = model(X_batch)
62         loss = loss_fn(y_pred, y_batch)
63         epoch_mse.append(float(loss))
64         # backward pass
65         optimizer.zero_grad()
66         loss.backward()
67         # update weights
68         optimizer.step()
69     mean_mse = sum(epoch_mse) / len(epoch_mse)
70     train_mse_history.append(mean_mse)
71     # validate model on test set
72     model.eval()
73     with torch.no_grad():
74         y_pred = model(X_test)
75         mse = loss_fn(y_pred, y_test)
76         mae = mae_fn(y_pred, y_test)
77         test_mse_history.append(float(mse))
78         test_mae_history.append(float(mae))
79
80 plt.plot(np.sqrt(train_mse_history), label="Train RMSE")
81 plt.plot(np.sqrt(test_mse_history), label="Test RMSE")
82 plt.plot(test_mae_history, label="Test MAE")
83 plt.xlabel("epochs")
84 plt.legend()
85 plt.show()

```

Further Readings

This section provides more resources on the topic if you are looking to go deeper.

APIs

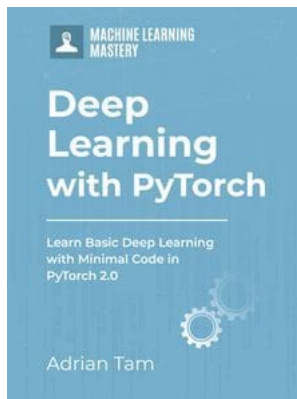
- [nn.L1Loss](#) from PyTorch documentation
- [nn.MSELoss](#) from PyTorch documentation

Summary

In this chapter, you discovered the importance of collecting and reviewing metrics while training your deep learning models. You learned:

- What metrics to look for during model training
- How to compute and collect metrics in a PyTorch training loop
- How to visualize the metrics from a training loop
- How to interpret the metrics to infer details about the training experience

Get Started on Deep Learning with PyTorch!



Learn how to build deep learning models

...using the newly released PyTorch 2.0 library

Discover how in my new Ebook:
Deep Learning with PyTorch

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation, training, evaluation, hyperparameter optimization*, and much more...

Kick-start your deep learning journey with hands-on exercises

SEE WHAT'S INSIDE

Share Share Post

More On This Topic



How To Investigate Machine Learning Algorithm Behavior



How to Update LSTM Networks During Training for Time...



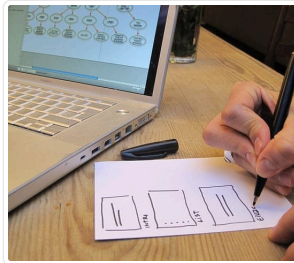
What To Do During Machine Learning Model Runs



5 Ways To Understand Machine Learning Algorithms...



Understand Machine Learning Algorithms By...



Understand Any Machine Learning Tool Quickly (even...



About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.
View all posts by Adrian Tam →

< Training a PyTorch Model with DataLoader and Dataset

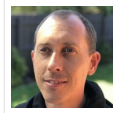
Managing a PyTorch Training Process with Checkpoints and Early Stopping >

No comments yet.

Leave a Reply

 Name (required) Email (will not be published) (required)

SUBMIT COMMENT



Welcome!

I'm *Jason Brownlee* PhD

and I **help developers** get results with **machine learning**.

[Read more](#)

Never miss a tutorial:



Picked for you:



PyTorch Tutorial: How to Develop Deep Learning Models with Python



LSTM for Time Series Prediction in PyTorch



Deep Learning with PyTorch (9-Day Mini-Course)



Calculating Derivatives in PyTorch



Building a Regression Model in PyTorch

Loving the Tutorials?

The [Deep Learning with PyTorch](#) EBook
is where you'll find the ***Really Good*** stuff.

>> [SEE WHAT'S INSIDE](#)



Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. [Visit our corporate website](#) to learn more about our mission and team.



[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved